codebase-memory-mcp 从入门到精通

项目	说明
教程主题	codebase-memory-mcp（下文简称 CBM）
覆盖版本	v0.12.x，以 v0.12.1（2026 年 10 月 16 日发布）为基准
覆盖平台	Windows、macOS、Linux
核实日期	2026 年 10 月 27 日
资料来源	项目仓库与官方文档（github.com/DeusData/codebase-memory-mcp）
读者假设	具备基本的命令行与 Git 使用经验；不要求预先了解 MCP 或图数据库

导读。 本教程按「入门篇、进阶篇、高级篇」三部分组织：第 1 章至第 3 章完成从概念认识到索引第一个仓库的完整起步；第 4 章至第 8 章覆盖日常使用的核心能力，包括智能体接入、三类检索、关系分析、图查询与可视化；第 9 章至第 12 章面向大型团队与复杂工程场景，涉及基础设施追踪、团队共享、配置调优与综合实战。文中命令默认在系统终端执行；MCP 工具调用以「工具名与参数」的形式给出，由编码智能体代为执行；除特别注明外，示例输出均为示意，实际内容取决于被索引的仓库。

第一部分 入门篇

第 1 章 认识 codebase-memory-mcp：本地代码智能与知识图谱

接手一个数十万行、横跨多个服务的仓库时，仅靠逐个文件阅读或文本搜索，往往难以回答「这个函数被谁调用」「这条接口路由由哪个服务实现」「这次改动会影响哪些模块」一类结构性问题。本章介绍 CBM 如何用一张本地知识图谱回答这些问题，并说明它的整体工作方式与四种运行形态，为后续安装与使用建立完整的概念基础。

1.1 CBM 的定位

codebase-memory-mcp 是一个开源的 MCP 服务器，它将代码、文档、配置与基础设施文件解析并索引为一张持久的本地知识图谱。知识图谱（knowledge graph）指以节点表示实体、以边表示实体之间关系的图结构数据。在 CBM 中，函数、类、文件、接口路由、容器镜像等都是节点；「调用」「导入」「部署」等关系则是连接节点的边。索引完成后，使用者可以查询这张图，也可以让编码智能体通过 MCP 直接向它提问。

MCP（Model Context Protocol，模型上下文协议）是 AI 客户端与外部工具服务器之间的一种开放通信协议。支持该协议的编码智能体（如 Claude Code、Codex CLI、OpenCode、Cursor 等）可以连接 CBM，把使用者的自然语言问题转换为对 17 个 MCP 工具的调用，再把图查询结果整理为答案。

CBM 的关键特征如下：

- **完全本地运行。**源码解析、图存储与查询都在本机完成，不需要托管服务，不需要 API 密钥，也不收集遥测数据。
- **原生实现，单一可执行文件。**后端以纯 C 编写，语法解析器直接内嵌，运行时不依赖 Docker 或任何语言运行时；标准发行版的可执行文件体积约为 330 MB。
- **语言覆盖广泛。**支持 162 种语言与文件格式的解析；对 Python、TypeScript/JavaScript/JSX/TSX、PHP、C#、Go、C、C++、Java、Kotlin、Rust、Perl 这 11 种语言提供 Hybrid LSP 类型解析（一种内嵌的语言级类型推断层，用于精确解析跨文件、跨模块的调用目标，详见第 7 章）。
- **可视化配套。**标准发行版内置 CBM Lens（开放测试版 v0.0.1），一个在浏览器中运行的代码图谱可视化界面，详见第 8 章。
- **许可与安全。**MIT 许可；每个发行版均经过 VirusTotal 扫描、SLSA 3 构建来源证明与签名验证（见第 11 章）。

1.2 为什么需要代码知识图谱

逐文件阅读代码的代价不仅是时间：当阅读者或 AI 智能体把文件内容载入上下文时，消耗的是语言模型的令牌（token，语言模型处理文本的计量单位）。结构性问题往往需要读大量文件才能回答，而图查询可以在毫秒级返回同等结论。

项目官方预印本（arXiv:2603.27277，2026 年 3 月测量）对 31 个仓库的评估显示：与逐文件探索相比，图查询将令牌消耗降低约 10 倍，工具调用次数减少约 2.1 倍。官方在 Apple M3 Pro 上发布的后端基准数据如下（仅供参考，实际耗时取决于硬件与仓库规模）：

操作	耗时	说明
Linux 内核全量索引	约 3 分钟	2800 万行代码、7.5 万文件，生成 481 万节点、772 万边
Linux 内核快速索引	1 分 12 秒	188 万节点
Django 全量索引	约 6 秒	4.9 万节点、19.6 万边
Cypher 图查询	小于 1 毫秒	关系遍历
名称搜索（正则）	小于 10 毫秒	SQL 前置过滤
死代码检测	约 150 毫秒	全图扫描加度数过滤
调用路径追踪（深度 5）	小于 10 毫秒	广度优先遍历

1.3 工作方式：索引、查询与智能体协作

CBM 是一个结构化分析后端：它负责构建与查询知识图谱，自身不包含生成式模型，智能的理解层由使用者的 MCP 客户端承担。一次典型交互如下：

```
使用者提问："什么代码调用了 ProcessOrder？"
        |
        v
编码智能体选择工具：trace_path(function_name="ProcessOrder", direction="inbound")
        |
        v
codebase-memory-mcp 执行图查询，返回结构化结果
        |
        v
智能体以自然语言整理并呈现调用链
```

也就是说，使用者不需要记忆工具名称与参数；以自然语言提问即可，智能体会自行选择合适的工具。掌握各工具的能力边界（第 5 章、第 6 章），有助于提出更容易被正确转换的问题。

1.4 四种运行形态

CBM 在同一可执行文件内提供四种运行形态，各自面向不同场景：

形态	启动方式	用途
MCP 服务器	由编码智能体自动拉起，或手动配置	供智能体调用 17 个 MCP 工具，是项目的核心形态
CLI（命令行界面）	`codebase-memory-mcp cli <工具名>`	不经智能体、直接在终端执行任一 MCP 工具，适合脚本化与验证
守护进程（daemon）	`codebase-memory-mcp daemon start`	后台常驻进程，承载文件监视、后台索引与 CBM Lens 界面
CBM Lens	浏览器访问守护进程提供的界面	以图形方式浏览图谱、阅读源码、管理架构决策

守护进程（daemon）指在后台持续运行的服务进程。CBM 的守护进程按账户共享：第一个使用它的 CBM 会话将其自动拉起，最后一个会话退出后自动停止；以 `daemon start` 手动启动的实例则持续运行直到显式停止。相关约束见 2.7 节。

1.5 适用与不适用场景

CBM 适合回答关于代码与部署的结构性问题：理解陌生功能、追踪调用链、评估变更影响、追溯部署关系、沉淀架构决策。它不适合、也不试图承担以下工作：运行或修改代码、执行测试（但可辅助选择测试范围，见 6.6 节）、替代人工审查。涉及删除索引、清理缓存等有数据损失风险的操作，本教程在相应位置明确标注后果。

第 2 章 安装与首次运行

安装是一切使用的前提，但安装方式的选择直接影响后续体验：脚本安装会同时完成智能体客户端配置，包管理器安装则便于纳入既有工具链。本章按平台给出安装步骤，解释安装脚本的行为与验证方法，并说明守护进程、更新与卸载的运行机制。

2.1 安装方式选择

方式	适用场景	特点
官方安装脚本（推荐）	个人开发机首次安装	一步完成下载、校验、安装与客户端配置，包含 CBM Lens
包管理器	已有 npm、pip、Homebrew、Scoop、AUR、Nix 或 Go 工具链	由包管理器负责缓存与升级
源码编译	需要定制或无发行版可用	需要 C/C++ 编译器、zlib 与 Node.js（用于构建界面）

无论采用哪种方式，得到的核心都是同一个原生可执行文件 `codebase-memory-mcp`（Windows 上为 `codebase-memory-mcp.exe`）。

2.2 系统要求

原生发行版对 macOS（Apple Silicon 与 Intel）、Linux（x86_64 与 ARM64）、Windows（x86_64 与 ARM64）均提供预编译包。运行时无外部依赖；磁盘空间需为可执行文件预留约 350 MB，另加索引数据占用的空间（随仓库规模增长，存储位置见 2.7 节）。

2.3 macOS 与 Linux：脚本安装

执行以下命令。其中 `curl` 的 `-f` 表示服务器错误时返回失败，`-s` 与 `-S` 组合为静默但保留错误信息，`-L` 表示跟随重定向；管道符 `|` 将脚本内容交给 `bash` 执行：

```
curl -fsSL https://raw.githubusercontent.com/DeusData/codebase-memory-mcp/main/install.sh | bash
```

脚本依次完成：下载与当前平台匹配的发行包、对照发布附带的 `checksums.txt` 校验 SHA-256 摘要（一种固定长度的文件指纹，用于确认文件未被篡改）、安装可执行文件、检测本机存在的编码智能体客户端并写入相应配置。安装完成后，重新启动编码智能体即可。

2.4 Windows：PowerShell 安装

Windows 的下载文件会带有 Mark-of-the-Web 标记（操作系统为来自网络的文件附加的安全标记），需先解除标记再执行：

```
# 1. 下载安装脚本
Invoke-WebRequest -Uri https://raw.githubusercontent.com/DeusData/codebase-memory-mcp/main/install.ps1 -OutFile install.ps1

# 2. （建议）用文本编辑器查看脚本内容
notepad install.ps1

# 3. 解除网络标记
Unblock-File .\install.ps1

# 4. 执行
.\install.ps1
```

若系统提示执行策略错误，可先执行 `Set-ExecutionPolicy -Scope Process Bypass`（仅对当前进程生效），或直接以 `PowerShell -ExecutionPolicy Bypass -File .\install.ps1` 运行。脚本支持两个选项：`--skip-config` 表示仅安装可执行文件、不配置智能体；`--dir=<路径>` 指定自定义安装位置。

Microsoft Defender 可能将发行包报告为 `Trojan:Script/Wacatac.B!ml`。项目方声明这是已知误报：VirusTotal 多引擎扫描中通常仅 Microsoft 一家报出，同类检测同样出现在 `gh`、llama.cpp、Godot 与微软自家 Go 工具链上。项目的 SECURITY.md 提供了自行核验发布件的方法（见 11.5 节）。

2.5 包管理器安装

渠道	命令
npm	`npm install -g codebase-memory-mcp@latest`
PyPI	`pip install -U codebase-memory-mcp`
Homebrew	`brew install codebase-memory-mcp`
Scoop	`scoop install codebase-memory-mcp`
AUR（Arch Linux）	`yay -S codebase-memory-mcp-bin` 或 `paru -S codebase-memory-mcp-bin`
Nix	`nix run github:DeusData/codebase-memory-mcp`
Go	`go install github.com/DeusData/codebase-memory-mcp/pkg/go/cmd/codebase-memory-mcp@latest`

说明：v0.12.1 修复了此前 npm、PyPI 与 Go 安装通道无法安装 v0.12.0 的问题（发行包超过包封装器旧的 256 MB 体积上限），并使 Homebrew 恢复安装当前版本、在 Linux 上改用静态构建。若使用更早版本的包封装器遇到安装失败，应先升级封装器本身。Nix 用户如需带界面的构建，可运行 `nix run github:DeusData/codebase-memory-mcp#codebase-memory-mcp-ui -- --ui=true --port=9749`。

2.6 验证安装

依次执行以下两项检查。第一条确认可执行文件可用并输出版本号；第二条模拟 MCP 客户端的标准输入输出握手：MCP 采用 stdio 传输，即服务器从标准输入读取请求、向标准输出写入 JSON 响应，因此送入一个空对象 `{}` 应得到 JSON 输出：

```
codebase-memory-mcp --version
```

```
echo '{}' | codebase-memory-mcp
```

第二条命令的输出为一行 JSON（示意输出，内容随版本变化）：

```
{"jsonrpc":"2.0","id":null,"error":{"code":-32700,"message":"Parse error"}}
```

得到任何合法 JSON 响应即表明二进制工作正常。若提示找不到命令，说明安装目录不在 PATH 环境变量（操作系统查找可执行文件的目录列表）中，可将其加入：

```
export PATH="$HOME/.local/bin:$PATH"
```

2.7 守护进程与会话协调

CBM 的守护进程承载长时后台服务：文件变更监视、共享索引任务与 CBM Lens 界面。它按账户共享，由 Claude Code、Codex、OpenCode 等所有已配置客户端共同使用。启动与查看：

```
codebase-memory-mcp daemon start --open --port=9749   # 启动守护进程并打开浏览器界面
codebase-memory-mcp daemon status                      # 查看运行状态
codebase-memory-mcp daemon stop                        # 停止守护进程
```

`--open` 表示启动后自动打开浏览器，`--port=9749` 指定本地服务端口（CBM Lens 默认端口）。以 `daemon start` 启动的实例将一直运行，直到执行 `daemon stop`；由会话自动拉起的实例则在最后一个会话断开后退出。

所有同时活动的 CBM 进程必须运行完全相同的版本、构建与缓存根目录。若设置了不一致的 `CBM_CACHE_DIR` 别名，系统会将其归一到同一根目录；真正不同的缓存根会在有进程活动时被拒绝。冲突记录写入 `${CBM_CACHE_DIR}/logs/daemon-conflicts.ndjson`。守护进程日志位于 `${CBM_CACHE_DIR}/logs`（默认 `~/.cache/codebase-memory-mcp/logs`）：

文件	内容
`cbm-daemon.log`	守护进程生命周期、监视与索引、界面、错误事件
`daemon-conflicts.ndjson`	版本不匹配、缓存根冲突等准入拒绝记录
`activation-events.ndjson`	安装、更新、卸载的进度与结果

2.8 更新与卸载

CBM 的更新从安装脚本发起，而不是由运行中的二进制自行完成。原因在于 Windows 不允许运行中的程序替换自身文件，且项目方有意让二进制保持单一职责。执行 `codebase-memory-mcp update` 会校验参数并打印确切的更新命令：

```
# macOS 与 Linux
bash "<安装目录>/install.sh"
```

```
# Windows
powershell -File "<安装目录>\install.ps1"
```

安装脚本在安装时被放置在可执行文件旁，且是幂等的（重复执行结果一致），因此重新运行安装脚本即是更新。通过 npm 或 pip 安装的使用者则用包管理器更新（`npm install -g codebase-memory-mcp@latest` 或 `pip install -U codebase-memory-mcp`）。

卸载命令为：

```
codebase-memory-mcp uninstall
```

该命令移除由它写入的智能体配置项、技能、钩子与已安装二进制。已建立的图索引默认保留，并在输出中列出缓存目录位置；交互终端下会单独询问是否删除，默认保留。`-y`（或 `--yes`）与 `-n`（或 `--no`）及非交互输入均保留索引，除非显式给出 `--delete-indexes`。预演模式不会改动任何文件：

```
codebase-memory-mcp uninstall -y --delete-indexes       # 连同全部项目索引一起删除
codebase-memory-mcp uninstall --dry-run --delete-indexes # 预演，不改动任何文件
```

注意：`--delete-indexes` 表示明确同意删除，其优先级高于 `--no`。保留索引时，卸载输出会打印缓存目录（默认 `~/.cache/codebase-memory-mcp`）与手动删除命令。位于二进制旁的安装脚本只被报告路径、不被删除，需要清理时按输出提示手动移除。

第 3 章 索引第一个仓库

知识图谱的价值建立在索引之上：只有把仓库解析为节点与边，后续的搜索、追踪与可视化才有数据来源。本章完成从建立第一个索引到提出第一个图查询的完整流程，并介绍自动索引、后台同步与索引范围控制。

3.1 建立索引

索引（indexing）指解析仓库源码并构建知识图谱的过程。索引入口有两个：MCP 工具 `index_repository`（通常由智能体调用），或 CLI 的一次性命令。以 CLI 为例：

```
codebase-memory-mcp cli index_repository --repo-path /absolute/path/to/repo
```

`--repo-path` 接受仓库的**绝对路径**（从文件系统根开始、不依赖当前目录的完整路径）；传入相对路径是索引失败最常见的原因。默认采用 `full` 模式，即包含语义分析的全量索引。

3.2 通过智能体索引

安装脚本已配置的客户端中，重启编码智能体后直接说：

```
Index this project
```

智能体会调用 `index_repository` 完成索引。若希望此后每次连接都自动索引新项目，可开启自动索引：

```
codebase-memory-mcp config set auto_index true
```

`auto_index` 默认关闭。开启后，新项目在首次连接时自动建立索引；可配置的文件数上限由 `auto_index_limit` 控制（见 11.1 节）。

3.3 查看项目与状态

```
codebase-memory-mcp cli list_projects
```

该命令列出已索引项目及其名称。后续所有 CLI 查询中的 `--project` 参数取值即为此处的名称。查看节点与边的统计信息：

```
codebase-memory-mcp cli list_projects --format json --detail stats
```

`--format json` 输出机器可读的载荷 JSON，`--detail stats` 附加节点数、边数与数据库大小。检查某个项目的索引状态与新鲜度：

```
codebase-memory-mcp cli index_status --project my-project
```

若需要移除某个项目的图谱数据，使用 `delete_project` 工具（MCP 调用）。注意其后果：项目对应的整张图被删除，且此操作不可撤销，但仓库中的源码文件不受任何影响。

3.4 大仓库的异步索引

同步索引会阻塞直到完成；部分 IDE 客户端对单次工具调用设有固定时限，超时后客户端断开，可能导致索引任务被取消。对此，`index_repository` 提供异步模式。第一步启动任务并立即返回：

```
index_repository(repo_path="/absolute/path", async: true)
```

返回值中的 `state` 字段为 `queued`（排队中）或 `running`（执行中）。第二步轮询状态，传入与启动时相同的 `repo_path`（若启动时使用了自定义 `name`，也需一致）：

```
index_repository(repo_path="/absolute/path", status: true)
```

`state` 的全部取值为 `queued`、`running`、`cancelling`、`succeeded`、`failed`、`cancelled`，轮询直至出现终态。异步任务由守护进程承载：客户端断开不会终止任务，只有停止守护进程才会终止。注意两点：`async` 与 `status` 互斥；CLI 的一次性 `--async` 调用在守护进程不存在且没有其他会话存活时会被拒绝（任务会随命令退出而消亡），此时应先执行 `daemon start` 或保持一个 MCP 会话连接。

3.5 索引模式

模式	用途
`full`	默认模式，全量索引并包含语义分析
`moderate`	过滤后的索引，保留语义分析
`fast`	过滤后的索引，不包含语义分析
`cross-repo-intelligence`	在多个已索引项目之间建立跨仓库连接（见 10.2 节）

时间紧张的场合可先用 `fast` 或 `moderate` 模式获得结构骨架，再按需重建 `full` 索引。

3.6 提出第一个问题

索引完成后即可查询。最直接的方式是图查询（查询语言见第 7 章）：

```
codebase-memory-mcp cli query_graph --project my-project --query 'MATCH (f:Function) RETURN f.name LIMIT 5'
```

该查询返回图中前 5 个函数的名称（示意输出，取决于仓库内容）：

```
f.name
------
parse_config
load_plugin
main
init_logger
render_template
```

也可以用智能体提问，例如「这个项目的主要模块有哪些」「入口函数在哪里」。智能体会选择 `get_architecture`、`search_graph` 等工具完成回答。

3.7 自动索引与后台同步

与索引自动化相关的设置有四个，均通过 `config set` 修改：

设置	默认值	含义
`auto_index`	`false`	MCP 会话首次连接时自动索引新项目
`auto_index_limit`	配置值	自动索引适用的文件数上限
`auto_watch`	`true`	允许活动 MCP 会话注册项目监视；设为 `false` 将移除全部监视
`watcher_enabled`	`true`	是否启动后台监视线程；该值在守护进程启动时读取，修改后需执行 `daemon stop` 并重启守护进程
`watch_non_git`	`false`	是否也轮询非 Git 仓库根目录

后台监视（watch）的行为有明确边界：被监视的项目默认仅限 Git 仓库；已索引项目的文件变更会被自动检测并增量重建索引；CLI 索引与失败的 MCP 索引不会注册监视。`index_status` 输出中的 `watch` 对象会报告项目是否被监视及上次扫描时间。

3.8 控制索引范围

索引器按三层规则决定哪些文件参与索引，优先级从高到低依次为：内置硬编码模式（如 `.git`、`node_modules` 等目录，始终忽略）、仓库的 `.gitignore` 层级、项目专属的 `.cbmignore` 文件。`.cbmignore` 使用与 `.gitignore` 相同的语法，用于在不影响 Git 的前提下排除 CBM 不需要的文件。例如，在仓库根目录创建：

```
# .cbmignore — gitignore 语法，仅对 CBM 索引生效
dist/
**/generated/
```

符号链接（symlink，指向另一路径的文件系统条目）始终被跳过。完整语法、优先级与取反语义见仓库内文档 `docs/cbmignore.md`，以官方文档为准。

第二部分 进阶篇

第 4 章 接入编码智能体

CBM 的设计目标是让编码智能体在日常工作中直接利用图谱，而不是每次都由使用者手工执行查询。本章说明 `install` 命令为各客户端写入了哪些内容、如何手动配置 MCP、三级智能体配置（Scout、Verify、Auditor）的分工，以及钩子如何在不干扰正常工具调用的前提下补充图谱上下文。

4.1 install 命令自动完成的工作

安装脚本最后会调用原生 `install` 命令，检测本机的编码智能体客户端并写入各自的集成内容。目前支持 45 个自动或条件式客户端集成面（其中 39 个自动检测，6 个需要条件或显式指定）。写入的内容分为五类：

组件	作用
MCP 配置	将客户端连接到本地服务器及其 17 个工具
指令与技能	说明图谱优先的探索方式、索引新鲜度与覆盖检查、何时直接阅读源码
Scout、Verify、Auditor 配置	提供三级图查询智能体（见 4.4 节）
会话与压缩钩子	在支持的客户端生命周期事件上恢复图谱上下文
搜索与读取钩子	为支持的搜索补充相关图谱符号，并在读取文件后标注覆盖缺口

安装过程遵守明确边界：不开启实验特性开关，不启用插件、YOLO 模式、全局权限绕过或第三方指令信任。

4.2 主流客户端的集成方式

下表摘录常见客户端的集成要点，完整矩阵见仓库 README 的 Multi-Agent Support 一节：

客户端	MCP 配置位置	额外集成
Claude Code	`~/.claude.json`	技能、三个图查询智能体、会话与工具钩子
Codex CLI	`$CODEX_HOME/config.toml`	`AGENTS.md` 托管指针、技能、三个只读智能体；钩子需经 `/hooks` 审阅信任
Gemini CLI	`.gemini/settings.json`	`GEMINI.md`、三个子智能体、工具前后钩子
OpenCode	`$OPENCODE_CONFIG` 或全局配置	`AGENTS.md`、技能、三个默认拒绝的只读智能体、压缩后重注入
Cursor	`.cursor/mcp.json`	技能与三个父级交接智能体（不安装上下文钩子）
VS Code	平台 `Code/User/mcp.json`	`~/.copilot/skills`、三个只读智能体、会话钩子
Windsurf	`~/.codeium/windsurf/mcp_config.json`	常开的 `global_rules.md`
Zed	平台 `settings.json`（JSONC）	`AGENTS.md` 与共享技能
Aider	无 MCP 配置	经 `.aider.conf.yml` 写入 `CONVENTIONS.md`

4.3 手动 MCP 配置

不使用自动配置时，可在客户端的 MCP 配置文件中手动添加。以通用 JSON 格式为例：

```
{
  "mcpServers": {
    "codebase-memory-mcp": {
      "command": "/path/to/codebase-memory-mcp",
      "args": []
    }
  }
}
```

`command` 必须是可执行文件的**绝对路径**。保存后重启编码智能体，在客户端的 MCP 面板中确认服务器出现且工具列表可见。

4.4 Scout、Verify 与 Auditor 三级配置

在支持自定义智能体格式的客户端中，安装器会写入三个具有明确分工的图查询智能体：

层级	名称	行为约定
第 1 级	Scout	约 3 至 4 次窄范围调用的快速探索，只做肯定性、临时性结论；不作「不存在」「影响已穷尽」「死代码」类断言
第 2 级（默认）	Verify	面向任务搜集图谱证据，精确核对源码，对引用的每个文件做路径覆盖，负面结论前先确认范围覆盖
第 3 级	Auditor	有界范围、当前索引代次、完整相关分页、更广的关系核对，并显式报告未能解决的限制

每一级都会对其证据路径批量执行 `check_index_coverage`（见 5.7 节）。覆盖检查的干净结果只意味着「没有记录在案的缺口」，不是完整性的证明。不更新用户修改过的智能体定义是安装器的明确策略：只有与上一版逐字节一致的 Verify 定义才会被迁移替换。

4.5 钩子的行为边界

由 CBM 安装的钩子（hook，即在客户端生命周期事件上自动注入上下文的机制）遵循两条原则：仅注入上下文（context-only），且失败时放行（fail-open）。也就是说，钩子永远不会拒绝、拦截或替换原始的工具调用。以 Claude Code 为例：`PreToolUse` 钩子观察 `Grep`、`Glob`、`Bash` 调用，注入匹配的图谱符号作为附加上下文；`Read` 之后注入目标文件的覆盖提示。Codex 用户需在 `/hooks` 中审阅并信任已安装的钩子；钩子定义变更会改变信任哈希，更新后可能需要重新信任。

4.6 高效的提问方式

智能体会把自然语言问题转换为工具调用；问题的结构越接近图查询的形态，转换越准确。下表给出常见问题与预期工具的对应关系：

使用者的提问	预期工具
「谁调用了 parseConfig？」	`trace_path`（入站方向）
「这个参数的值从哪里来？」	`trace_path`（`data_flow` 模式）
「这次未提交的改动影响哪些符号？」	`detect_changes`
「这个接口由哪个服务实现？」	路由匹配相关的图查询
「这个文件里有哪些定义？」	`get_file_outline`
「项目的整体结构是什么？」	`get_architecture`

4.7 长会话中的上下文维持

编码智能体的长会话会经历压缩（compaction，即客户端为控制上下文长度而压缩历史的过程）。对于支持的客户端，CBM 的钩子会在压缩后重新注入图谱上下文，使后续对话仍然知道可用的项目名、索引状态与工具用法。在不支持稳定上下文事件的客户端上，安装器以持久化指令文件代替钩子：智能体在每次委派子任务时，把项目名、限定名称与调用链证据传递给子智能体。

第 5 章 三类检索：结构化、全文与语义搜索

在一个已索引的仓库里找到「正确的符号」，是后续一切分析的基础。CBM 提供三种互补的检索方式：按名称与结构的精确搜索、按文本内容的全文搜索、按含义相近的语义搜索。本章逐一介绍三者的适用场景与参数，并补充读取源码、文件大纲与项目总览的工具。

5.1 先看图的结构

对不熟悉的仓库，建议先调用 `get_graph_schema` 了解图谱的构成。该工具返回节点与边的数量、关系模式以及每种标签的属性定义。官方文档将其定位为「最先运行的工具」。

5.2 结构化搜索：search_graph

`search_graph` 按名称模式、标签、度数与文件范围过滤图节点。核心参数：

参数	含义
`name_pattern`	名称的正则表达式（regular expression，一种描述字符串模式的表达式）匹配，如 `.*Handler.*`
`label`	节点标签过滤，如 `Function`、`Class`、`Route`
度数上下限	按调用关系的最小与最大度数过滤（度数指与节点相连的边数）
文件范围	将搜索限制在指定文件内
`offset` / `limit`	结构化结果的分页参数

CLI 调用示例：

```
codebase-memory-mcp cli search_graph --project my-project --name-pattern '.*Handler.*' --label Function
```

需要机器可读输出时追加 `--format json`：

```
codebase-memory-mcp cli search_graph --project my-project --label Function --format json
```

5.3 全文搜索

全文检索基于 SQLite FTS5 与 BM25 排名算法（一种结合词频与文档长度的经典文本相关性排序算法），分词器 `cbm_camel_split` 能识别 camelCase 与 snake_case 的边界，因此搜索 `parse_config` 也能命中 `parseConfig`。

若需要的是「在文件里按行找文本」，使用 `search_code`：它是图增强的 grep（grep 指逐行匹配文本内容的搜索工具），只在已索引文件范围内执行，并按相关性对结果排序。其分页参数包括：`result_limit` / `result_offset`（控制排序后的结果行）、`raw_limit` / `raw_offset`（控制原始匹配行）、`directory_limit` / `directory_offset`（控制目录摘要）。

5.4 语义搜索

语义搜索回答「含义相近」的检索需求，例如按功能描述找实现。它不是独立工具，而是 `search_graph` 的 `semantic_query` 参数：向量搜索覆盖整张图，嵌入模型（embedding，将文本映射为向量的技术）`nomic-embed-code` 直接编译进二进制，无需 API 密钥，也不依赖 Ollama 或 Docker。评分由 11 种信号组合而成，包括 TF-IDF、API 与类型签名、AST 概要、数据流、MinHash、模块邻近度与图扩散等。语义结果的独立分页参数为 `semantic_offset` / `semantic_limit`，与结构化结果的分页相互独立。

5.5 读取源码与大纲

- `get_code_snippet`：按**限定名称**（qualified name，形如 `<项目名>.<路径各段>.<符号名>` 的唯一标识）读取一个函数的源码。限定名称先用 `search_graph` 查得，再传入本工具。
- `get_file_outline`：返回单个文件（相对仓库根的路径）内声明的顺序大纲，可按标签过滤并分页。适合在深入阅读前了解文件的结构组成。

5.6 项目总览：get_architecture

`get_architecture` 默认返回紧凑的项目概览：语言构成、包结构、路由、热点、聚类与架构决策记录。需要更细的视角时，通过 `aspects` 参数选择 `routes`（路由）、`hotspots`（热点）、`dependencies`（依赖）、`clusters`（聚类）或 `cycles`（循环依赖）。`system_structure` 投影（CBM Lens 界面使用的系统结构视图）必须单独请求。

5.7 核对覆盖情况

把「图里查不到」当作「代码里不存在」之前，应先调用 `check_index_coverage`，传入确切的路径或范围，确认相关文件已索引且结果新鲜。需要强调项目方的原话：干净的结果只表示「没有记录在案的缺口」，不是完整性的证明。

第 6 章 关系分析：调用链追踪、变更影响与死代码

找到符号之后，真正回答工程问题的是关系：调用链说明「谁用了它」，变更分析说明「改了它会影响谁」，死代码候选说明「可能没人用它」。本章介绍 CBM 的四个关系分析工具与实验性的测试选择能力。

6.1 调用链追踪：trace_path

`trace_path`（别名 `trace_call_path`）沿图中的关系边追踪路径。核心参数：

参数	含义
`function_name`	起始符号名称（建议先用 `search_graph` 确认准确拼写）
`direction`	`inbound`（谁调用它）、`outbound`（它调用谁）、`both`（双向）
`depth`	追踪深度，取值 1 至 15
`mode`	路径类型，见下表

`mode` 的取值与用途：

模式	用途
`calls`（默认）	调用关系路径
`data_flow`	数据流路径：实参到形参的映射与字段访问链
`cross_service`	跨服务路径：HTTP、gRPC、消息通道等跨服务调用
`topology`	拓扑路径：Kubernetes 对象、路由、控制器及其部署关系（从资源或路由节点出发）
`deploy`	部署路径：CI 任务、脚本或 Make 规则运行并部署了什么
`run`	运行路径：Service 到负载到镜像到容器启动的程序

CLI 示例（双向追踪名为 `Search` 的函数）：

```
codebase-memory-mcp cli trace_path --project my-project --function-name Search --direction both
```

测试代码默认从结果中排除。追踪返回 0 条结果时，最常见的原因是名称拼写不完整，应先执行结构化搜索确认精确名称。

6.2 变更影响分析：detect_changes

`detect_changes` 将未提交的 Git 变更映射为受影响的符号清单，并给出风险分级（risk classification，按影响的广度与位置对变更危险程度分类），同时汇总模块级摘要。对智能体辅助的调查场景，传入 `scope: "tests"` 可额外报告测试影响。结果按变更文件、受影响符号与模块摘要分别分页，并附带与快照绑定的续读游标，保证多次读取之间不重不漏。

6.3 快照对比：compare_graphs

`compare_graphs` 对比两个已索引的快照（snapshot，某次索引完成时的图状态），报告基准与目标之间节点与边的增删。适合回答「这个版本相比上个版本，哪些函数与调用关系出现了或消失了」。

6.4 死代码候选

死代码（dead code）候选指图中没有已索引调用者的函数，入口点（entry point，程序启动或框架约定的调用起点）会被排除。CLI 查询约需 150 毫秒；也可用 Cypher 表达同样的判断：

```
MATCH (f:Function)
WHERE NOT EXISTS { (f)<-[:CALLS]-() }
RETURN f.name
```

删除任何「死代码」之前必须人工核对三类情况：动态分发（通过函数指针、反射或接口的间接调用在静态图中不可见）、覆盖率缺口（未索引的文件可能包含调用方）、框架约定（路由处理器、回调等由框架按约定调用的代码）。

6.5 实验性 Smart CI 与测试选择

Smart CI 是实验性能力，分两层：CI 任务选择按变更文件列表挑选需要运行的 CI 任务；测试选择组合静态可达性、变更的测试与已核准的逐测试覆盖数据，为受支持的 C/C++ 测试框架选出需要运行的测试子集。命令行入口为 `test-impact select`（产出带理由的选择）与 `test-impact publish`（产出作为基线的图与覆盖数据包）。证据缺失或不完整时，选择会自动放宽或回退为全量运行；仅有静态可达性而无可信覆盖数据时，任何测试套件都不会被省略。

建议：先以 report-only 影子模式运行并验证选择结果，再考虑将其接入门禁。测试收窄目前仅支持符合约定或声明等价的 C/C++ 测试框架，并非对 CBM 支持的所有语言通用。MCP 侧的测试影响通过 `detect_changes` 的 `scope: "tests"` 暴露。详见仓库内文档 `docs/SMART_CI.md`。

第 7 章 Cypher 查询与图数据模型

内置工具覆盖常见问题，但「图」的完整能力需要一种查询语言来释放。本章介绍 CBM 的图数据模型（节点标签、边类型与限定名称），以及 `query_graph` 支持的 Cypher 查询子集，使读者能够表达任意只读的结构化问题。

7.1 图数据模型

节点标签。 图中的节点按实体类型标注，主要标签包括：

类别	标签
结构	`Project`、`Package`、`Folder`、`File`、`Module`、`Branch`
代码实体	`Class`、`Struct`、`Interface`、`Trait`、`Enum`、`Type`、`Function`、`Method`、`Field`、`Variable`、`Constant`、`Macro`、`Decorator`
接口与通信	`Route`（接口路由）、`Channel`（消息通道）、`Process`（被启动的外部程序）
配置与基础设施	`EnvVar`、`Resource`（Kubernetes 等资源）、`Chart`（Helm 图表）、`Image`（容器镜像）、`Table`
文档	`Section`（文档章节节点）

核心边类型。 常用的代码关系边有：`CALLS`（调用）、`CALL_REFERENCE`（以值引用方式使用可调用对象且目标唯一）、`USAGE`（使用了标识符但无法证明唯一目标）、`IMPORTS`（导入）、`DEFINES` / `DEFINES_METHOD`（定义）、`IMPLEMENTS`（实现）、`INHERITS`（继承）、`OVERRIDE`（覆写）、`DECORATES`（装饰）、`DATA_FLOWS`（数据流）、`RAISES` / `THROWS`（抛出异常）、`TESTS` / `TESTS_FILE`（测试关系）、`MENTIONS`（文档提及代码）、`SIMILAR_TO`（近重复代码，基于 MinHash 与 Jaccard 相似度）与 `SEMANTICALLY_RELATED`（语义相关，评分不低于 0.80）。基础设施相关的边见第 9 章，跨仓库的 `CROSS_*` 边见第 10 章。

`MENTIONS` 的绑定规则值得注意：仅当文档注释中的引用（如 C# 的 `<see cref>`、`<seealso cref>`）在语言自身的查找规则下恰好命中一个定义时才建立边；未解析的引用按原因计数并出现在 `index_status` 的 `doc_links` 中。

限定名称。 `get_code_snippet` 等工具使用限定名称定位符号，格式为 `<项目名>.<路径各段>.<符号名>`。限定名称先用 `search_graph` 发现，再传入其他工具。

7.2 支持的 Cypher 子集

`query_graph` 执行只读的 openCypher 子集（Cypher 是图数据库的声明式查询语言；openCypher 是其开放规范）。支持的内容如下：

- **子句**：`MATCH`、`OPTIONAL MATCH`、多个 `MATCH`、`WHERE`、`WITH`（含 `WITH … WHERE`）、`RETURN`、`ORDER BY`、`SKIP`、`LIMIT`、`DISTINCT`、`UNWIND`、`UNION` / `UNION ALL`、`CASE`。
- **模式**：带标签的节点、标签交替（如 `(n:Class|Struct)`）、关系类型与方向、可变长度路径（如 `[*1..3]`）、内联属性映射（如 `{name: 'main'}`）。
- **WHERE 条件**：比较与逻辑运算符、`IN`、`CONTAINS`、`STARTS WITH`、`ENDS WITH`、`IS [NOT] NULL`、正则匹配 `=~`、标签测试 `n:Label`，以及单跳存在性检查 `EXISTS { (n)-[:TYPE]->() }`。
- **聚合函数**：`count`（含 `DISTINCT`）、`sum`、`avg`、`min`、`max`、`collect`。
- **标量函数**：`labels`、`type`、`id`、`keys`、`properties`、`toLower`、`toUpper`、`toString`、`toInteger`、`toFloat`、`toBoolean`、`size`、`length`、`trim`、`ltrim`、`rtrim`、`reverse`、`coalesce`、`substring`、`replace`、`left`、`right`。

超出该子集的内容（写入类子句、`MERGE`、`CALL`、不支持的函数、列表与映射字面量、推导式、路径函数、参数等）会以明确的 `unsupported …` 错误失败，而不是返回空结果。这一错误行为使「查询失败」与「查询成功但无结果」可以明确区分。

7.3 实用查询示例

以下示例均使用已支持的子集。直接调用 CLI：

```
codebase-memory-mcp cli query_graph --project my-project --query "MATCH (f:Function)-[:CALLS]->(g) WHERE f.name = 'main' RETURN g.name"
```

示例 1：查找直接调用了名为 `main` 的函数的所有函数。

```
MATCH (f:Function)-[:CALLS]->(g) WHERE f.name = 'main' RETURN f.name, g.name
```

示例 2：沿调用链向前追溯三层，找出所有能到达 `main` 的函数。

```
MATCH (a:Function)-[:CALLS*1..3]->(b:Function {name: 'main'})
RETURN DISTINCT a.name
```

示例 3：统计各文件的函数数量，按数量降序排列。

```
MATCH (file:File)-[:DEFINES]->(f:Function)
RETURN file.path, count(f) AS functions
ORDER BY functions DESC
```

示例 4：查找没有任何调用者的函数（死代码候选，同 6.4 节）。

```
MATCH (f:Function)
WHERE NOT EXISTS { (f)<-[:CALLS]-() }
RETURN f.name
```

示例 5：查找继承了指定类的所有类。

```
MATCH (c:Class)-[:INHERITS]->(p:Class {name: 'BaseHandler'})
RETURN c.name
```

7.4 文档与代码的连接

文档参与图谱的方式与代码相同：Markdown、reStructuredText 与 AsciiDoc 被解析为可导航的 `Section` 章节节点；带文本层的 PDF 被解析为可搜索的文本页（不做 OCR，即不识别纯图像型 PDF）。当文档中的引用能够解析到代码定义时，建立 `MENTIONS` 边；架构决策记录贡献状态、决策者与替代关系等元数据（管理接口见 8.6 节）。因此「找到解释」与「找到实现」可以在同一次图遍历中完成。需要核对某类文档或引用是否被索引时，使用 `check_index_coverage`。

第 8 章 CBM Lens 可视化导航

数字与文本形式的查询结果适合精确分析，而理解一个大型系统的空间结构时，可视化的效率更高。CBM Lens 是 CBM 的可选可视化配套（开放测试版 v0.0.1），随标准原生发行版提供：它把文件夹渲染为嵌套的平台块、把文件与符号渲染为可选择的单元、把已索引的关系渲染为箭头。本章介绍其启动方式、六个工作区与本地代码问答能力。

8.1 启动与添加项目

启动本地服务并自动打开浏览器：

```
codebase-memory-mcp daemon start --open --port=9749
```

浏览器打开 `http://127.0.0.1:9749`。服务只监听本机回环地址（loopback，即仅本机可访问的网络接口）。手动启动的守护进程会保持界面可用，结束使用后执行 `daemon stop`。在 MCP 会话中，也可以用 `--ui=true --port=9749` 启用同一共享界面。

添加首个项目：打开项目菜单，选择 **Add project index**（快捷键 `Alt+P`），选择本地仓库文件夹并为索引命名，然后开始索引。文件保持原位，不会被移动或复制。索引完成后打开项目，从 **Architecture** 获得总览，或从 **Explore** 开始阅读文件。需要分享某个视图时，使用形如 `?project=<名称>&workspace=architecture` 的地址参数，`workspace` 的可用取值为 `explore`、`galaxy`、`adr`、`coverage`、`system` 与 `architecture`。

8.2 六个工作区

工作区	用途
Explore	导航文件树、在编辑器中阅读源码、查看文件影响与符号详情
Galaxy	以三维图形式搜索、聚焦图谱节点并逐层展开关系
Architecture	浏览文件夹地图、入口点、路由、热点、系统结构与行为路径
ADR	阅读与编辑项目的架构决策记录
Coverage	检查已索引、部分解析、跳过与排除的路径及新鲜度信息
System	查看守护进程活动、索引与日志，可按项目、严重级别或文本过滤

8.3 Explore：阅读源码并理解上下文

Explore 提供集成开发环境式的工作区：左侧文件树，右侧 Monaco 源码阅读器（Monaco 是 VS Code 使用的浏览器端代码编辑器组件，提供语法高亮）。打开文件后，**File impact** 面板列出依赖当前文件的其他文件（即改动当前文件会影响哪些使用者）；**Code details** 面板显示光标所在符号的调用者、被调用、引用与相关测试。选中一段代码即可向本地问答发起提问（见 8.8 节）。

8.4 Galaxy：三维图谱探索

Galaxy 以三维空间呈现知识图谱。核心操作：

- 按快捷键 `Cmd+K`（macOS）或 `Ctrl+K`（Windows、Linux）打开全局搜索，按名称或路径检索符号、文件与文件夹，包括尚未加载到场景中的节点。
- 搜索并聚焦节点后，逐层加载其相邻关系；可切换到层级视图，或沿一条路径跟踪到目标节点并打开其源码。
- 边上的脉冲动画表示关系的方向，不代表真实执行；系统遵循「减少动态效果」的系统偏好设置时脉冲会暂停。
- 覆盖阴影（coverage shadow）标记已加载图周围已知的索引缺口。

后退与前进按钮可在历史视图之间切换。视图中会显示加载上限与省略计数；缺失的边可能源于索引或显示限制，得出结论前应检查 Coverage 工作区与源码。

8.5 Architecture：系统结构总览

Architecture 工作区包含五个视图：

- **Overview**：以三维地图遍历源码文件夹与文件，可选源码体量高度或统一高度、按语言或类别着色，并按关系类型过滤；切换到 **Entry points** 可从某个起始符号沿静态调用图追踪。
- **Routes**：在服务地图与端点连接之间切换；部署元数据与已索引的路由证据分别显示，未解析的注册仍保持可见。
- **Hotspots**：基于扇入（fan-in，被引用次数）与复杂度等测量信号定位经常被引用的区域；引力井效果帮助定位热点，但它不是代码质量评分。
- **System structure**：检查组件依赖、展开分组、隔离循环依赖，并可从连接回溯到源码证据。
- **Behavior**：选择一个起点与可达目标，检查静态调用路径及其源码语句。这些路径展示的是可能存在的关系，不是记录下来的执行顺序；需要运行时结论时应引入单独采集的轨迹（见 10.3 节）。

三维视图中，拖拽可环绕视角，滚轮缩放，点击节点或边查看其证据；不同关系类型以颜色区分。每个面板都会注明其事实来源，以及索引未能记录、以缺口形式呈现的部分。

8.6 ADR：架构决策记录

ADR（Architecture Decision Record，架构决策记录）以结构化文档保存重要的技术决策。ADR 工作区为每个项目维护一份未保存草稿；编辑器接受最多 8000 UTF-8 字节。保存是显式操作：保存修改的是**与索引一起存储的决策文档**，不是仓库中的任何文件。该文档与 MCP 工具 `manage_adr` 共享：`get` 读取、`update` 替换整个文档、`set_sections` 只重写指定名称的章节并逐字节保留其余内容、`sections` 列出标题。同一章节被重复写入时为无操作，因此响应丢失后的重试不会造成内容重复。

8.7 Coverage 与 System：核对索引与排查问题

Coverage 工作区列出已索引、部分解析、跳过与被排除的路径，以及可用的时效信息；把图查询的空结果当作「代码中不存在」之前，应先在此核对。System 工作区展示守护进程活动、已建立索引与日志，支持按项目、严重级别或文本过滤。前端错误与失败请求记录在本地，可通过 System 日志查看；`${CBM_CACHE_DIR}/logs/ui.log` 同时保留为轮转的 JSONL 导出文件。

8.8 本地代码问答

CBM Lens 内置一个完全在浏览器中运行的本地问答面板，用于解释当前正在查看的代码。启用步骤：

1. 点击右上角的机器人图标打开智能体配置。
2. 选择本地模型。默认的 Qwen2.5 Coder 0.5B 需下载约 567 MB；可选 Qwen3 0.6B、LFM2.5 1.2B 与 Qwen3.5 2B 等更大的模型。全部模型要求浏览器支持 WebGPU 及 `shader-f16` 特性。
3. 点击 **Download & load**。模型文件来自 Hugging Face，推理在浏览器内完成，不经过任何托管聊天服务；已缓存模型可用 **Load model (cached, no download)** 直接加载。
4. 提问。输入框会显示将随问题附带的文件或选区；希望聚焦答案时选择更小的范围。

问答有两类答案：调用者与被调用者一类的问题（例如「谁调用了 cbm_pipeline_run？」）直接从已加载的图谱作答，不需要模型参与，回答中会注明；选中符号后，面板列出其已索引事实与代码行，模型补充一句解释。若模型声称了源码未显示的内容，该句会被省略；模型生成的文字均带有标注。

使用限制需要知悉：聊天历史按项目保存在当前浏览器中 24 小时；每条消息使用阅读器中当前打开的源码，不会静默累积早前的源码快照；提示过长时先舍去较旧的消息，仍放不下时界面会要求缩小选区，而不是裁剪代码。该面板只能解释给出的上下文，不能编辑文件或执行工具。默认模型规模较小，措辞可能不严谨，尤其在英语以外语言中；其回答应与源码核对。

第三部分 高级篇

第 9 章 基础设施图谱：容器、Kubernetes、Terraform 与部署追踪

「这个服务由什么部署？」「运行它的容器镜像是什么？」「这行配置会作用到哪些代码？」这类问题横跨源码与基础设施，通常需要人工在多个文件类型之间来回跳转。CBM 将受支持的基础设施形态解析为图节点，并与它们构建、配置、运行的源码建立关系。本章介绍支持的形态、三种追踪模式与关键边类型的含义。

9.1 支持的基础设施形态

基础设施即代码（Infrastructure as Code，IaC，指以声明式文件描述基础设施的实践）在 CBM 中按三组形态接入：

分组	支持的形态
容器与集群	Dockerfile（含 `Dockerfile.*`、`*.Dockerfile`、`Containerfile`）、docker compose 服务、Kubernetes 资源、Helm 图表与发行、Kustomize 覆盖与补丁
供应与配置	Terraform/OpenTofu 资源与模块、tfvars 覆盖、Terragrunt 依赖、Ansible 任务与处理器、支持的配置键与环境变量访问
构建与交付	shell 命令、Make 目标、构建镜像或调用仓库脚本、部署资源的受支持 CI 步骤

9.2 三种追踪模式

`trace_path` 的三个基础设施模式分别回答三类问题：

模式	回答的问题	起点
`run`	这个服务运行的是什么？	Service、工作负载或 compose 服务
`deploy`	这个 CI 任务、脚本或 Make 规则部署了什么？	任务（出站）或图表（入站）
`topology`	这些 Kubernetes 对象、路由与控制器之间的关系是什么？	`Resource` 或 `Route` 节点

以 `run` 模式为例，可追溯的完整链条是：Service → 工作负载 → 镜像 → Dockerfile → 容器启动的程序，并附带每个容器的环境变量来源。这一链条在 v0.12.1 之后对 Helm 同样成立：HelmRelease → Chart → Deployment → Image 已成为一条可遍历的路径。

9.3 关键边类型解读

边类型	含义
`SPAWNS`	调用方启动另一个程序（如 `subprocess.run`、`exec.Command`、shell 命令、Make 配方、Dockerfile 的 `ENTRYPOINT` / `CMD`）；目标是按程序命名的 `Process` 节点
`INFRA_MAPS`	Kubernetes Service 到其选择器标签完全匹配的工作负载（同一命名空间内）
`CONNECTS_TO`	容器环境变量中的地址（如 `*_ADDR`、`*_HOST`、`*_URL` 键的值）指向的 Service 或 compose 服务；边保留键名，不保留取值
`DEPLOYS`	把清单、kustomization 或图表部署到集群的行为（Flux、Argo、`kubectl apply`、`helm install`、Terraform 的 `kubernetes_*` 资源等）
`DEPENDS_ON`	Helm 图表依赖、Dockerfile 基础镜像、模块依赖文件指向外部包等依赖关系
`CONFIGURES`	代码读取环境变量、容器设置环境变量、配置键与代码符号之间的精确名称关联；环境变量的取值永不存储
`OVERRIDE`	Kustomize 补丁、名称前后缀对基础对象的改写；Terraform 的 tfvars 覆盖同理

9.4 v0.12.1 的 Helm 与镜像改进

v0.12.1 之前，Helm 图表在图中是孤立节点。v0.12.1 起建立了三类连接：图表到其模板渲染出的每个 Kubernetes 对象（`DEPLOYS`，`via: chart`，子图表的对象归属子图表）；`values.yaml` 及其变体到图表（`CONFIGURES`）；图表到其对象字面命名的镜像（`DEPLOYS`，`ref: image`；模板化的镜像不命名任何镜像，也不会被猜测）。在 20 个基础设施仓库的测量中，无出边的图表从 260 个中的 238 个降至 109 个，新增 2511 条图表到对象的连接。

同版本还修正了镜像命名：未写明镜像仓库主机（registry，存放容器镜像的服务）的镜像不再被归入虚构的 `docker.io/library/...`，而是按源码书写形式保存并标记 `registry: unknown`；写明主机的保留原值。由于镜像身份发生变化，升级后每个既有项目的首次索引是一次性的全量重建。

9.5 局限与核对方法

基础设施追踪基于受支持源码形态的**静态关系**：动态取值、无法解析的模板、运行于集群外部的资源都可能造成缺口。项目方对 Helm 模板渲染的已知限制也有明示：每个模板文件独立渲染，而真实 Helm 渲染共享同一个 `.Values`，因此在某文件中改写 `.Values` 并在另一文件读取的图表仍可能显示错误的镜像。据此，基于图得出的部署结论应在 CBM Lens 的 Architecture 面板或源码中核对证据；缺失不等于不存在。

第 10 章 团队共享、跨仓库智能与运行时轨迹

单人使用之外，CBM 提供三类组织级能力：把索引基线分享给团队、把多个仓库的图谱连接起来、把运行时观察到的调用关系并入静态图。本章依次介绍三者的机制与使用边界。

10.1 团队共享快照

`.codebase-memory/graph.db.zst` 是存储在仓库源码旁的图快照（artifact，可分享的压缩工件），内容为知识图谱的 SQLite 数据库副本：去除索引、经 `VACUUM INTO` 压缩整理，再以 zstd 压缩（zstd 是一种高压缩率通用压缩算法；典型压缩比 8:1 至 13:1）。该工件默认不生成：`index_repository` 仅在传入 `persistence: true` 时写入，此后每次索引都会刷新它；本地索引的持久化不受此参数影响。

工作方式：

- **基线导入。**团队成员克隆包含该工件的仓库后首次索引时，CBM 先导入基线，再对其本地变更做增量索引，尽可能复用共享基线。
- **两级写入。**最优级（`zstd -9` 加索引剥离与 `VACUUM INTO`）由显式 `persistence: true` 的索引写入；快速级（`zstd -3`）由后续刷新（包括监视器的低延迟增量更新）使用。
- **合并策略。**首次导出会创建 `.codebase-memory/.gitattributes` 并写入 `merge=ours`。依赖它处理并发更新前，应先确认仓库的 Git 合并驱动配置。
- **提交节奏。**工件每次索引都会被重写，Git 会把每次重写存为一个完整新对象。每次保存都提交会使一个 20 MB 的文件膨胀为数 GB 的历史（官方记录有团队在约 350 次提交中累积约 6 GB）。建议按发布、里程碑或夜间任务等固定节奏提交，而不是每次保存都提交。
- **Git LFS 方案。**若确需每次提交都携带，从仓库根的 `.gitattributes` 追踪该文件（同时保留自动生成的内层文件以继续提供 `merge=ours`）：

  ```
  .codebase-memory/graph.db.zst filter=lfs diff=lfs merge=lfs -text
  ```

  只追踪 `.zst` 文件即可；`artifact.json` 体积小且携带模式版本。采用 LFS 前应权衡：GitHub 对 LFS 存储与带宽单独计量，其对象无法自行清理；且每位团队成员都需要执行 `git lfs install`，否则检出得到的是指针文件，完整性校验的导入会拒绝它并回退为全量重索引。
- **退出选项。**共享导出完全自愿，Git 提交与否同样由团队决定；将 `.codebase-memory/` 加入 `.gitignore` 即可让每位成员从零建立本地索引。

10.2 跨仓库智能

将多个仓库索引到同一存储后，跨仓库连接把服务之间的调用关系连成整体。执行模式为 `cross-repo-intelligence` 的索引：

```
index_repository(mode="cross-repo-intelligence", target_projects=["*"])
```

`target_projects` 限定参与连接的目标项目，`["*"]` 表示全部。生成的 `CROSS_*` 边（如 `CROSS_HTTP_CALLS`、`CROSS_GRPC_CALLS`、`CROSS_GRAPHQL_CALLS`、`CROSS_TRPC_CALLS`、`CROSS_ASYNC_CALLS`、`CROSS_CHANNEL`）把跨仓库的 HTTP、gRPC、GraphQL、tRPC、异步与通道调用连到对端。区分「从未连接」与「已连接但无匹配」依靠状态标记：`index_status`（及界面布局）中的 `cross_repo` 对象在从未运行过跨仓库连接时为 `{"status":"never_run"}`，运行过后变为 `"ran"` 并附带时间、结果、目标与 `CROSS_*` 边总数。该状态与边一同存储在项目自身索引中，重建索引会一并清除并回到 `never_run`。跨仓库架构总览可在多仓库之上汇总服务、路由与依赖。

兼容性说明：在模式变更（第 768 号模式）之前构建的索引会被跳过而不是中断整个运行，响应中以 `skipped_projects` 列出并提示重建。

10.3 运行时轨迹

静态调用图展示「可能发生的调用」；实际发生了什么，需要另行采集的运行时数据。`ingest_traces` 工具把观测到的调用路径或调用者与被调用者对存入图谱，并以运行标签（run label）分组。约束：两个端点都必须能解析到已索引的符号；无法解析的名称会在结果中报告。轨迹并入后，第 9 章 Behavior 视图等静态路径分析可以与真实执行证据互为印证。

第 11 章 配置调优、安全与诊断

默认配置适合大多数场景，但大型仓库、容器环境与团队规范会提出额外要求。本章汇总配置子命令、环境变量、自定义扩展名与资源限制，随后介绍发行版的安全验证机制与诊断方法，最后给出常见故障的排查对照。

11.1 config 子命令与关键设置

```
codebase-memory-mcp config list                          # 查看全部设置
codebase-memory-mcp config set auto_index true           # 会话开始时自动索引
codebase-memory-mcp config set auto_index_limit 50000    # 自动索引的文件数上限
codebase-memory-mcp config set auto_watch false          # 移除并禁用全部后台监视（默认 true）
codebase-memory-mcp config set doc_candidates false      # 超大仓库跳过文档与代码的候选评分
codebase-memory-mcp config set watcher_enabled false     # 彻底停止监视线程（默认 true）
codebase-memory-mcp config set watch_non_git true        # 同时轮询非 Git 项目根目录（默认 false）
codebase-memory-mcp config set index_max_files 250000    # 单次索引的源文件数上限
codebase-memory-mcp config set index_max_source_mb 16384 # 单次索引的源体积上限（MiB）
codebase-memory-mcp config reset auto_index              # 将某项设置恢复默认
```

要点：`doc_candidates` 的默认值为 `auto`（`full` 模式开启、`moderate` 与 `fast` 模式关闭）；两个 `index_max_*` 设置默认关闭，超出上限时整个索引尝试失败而非发布不完整的图，已在服务的旧索引保持不变。`watcher_enabled` 在守护进程启动时读取一次，修改后需执行 `daemon stop` 并重新连接才会生效；`auto_watch` 则在下一次刷新或注册请求时生效。完整的配置文件参考见仓库内文档 `docs/CONFIGURATION.md`。

11.2 环境变量

变量	默认值	说明
`CBM_CACHE_DIR`	`~/.cache/codebase-memory-mcp`	数据库与配置的存储目录；同一账户同一时间只能使用一个规范缓存根，切换前需关闭活动的 CBM 会话
`CBM_ALLOWED_ROOT`	未设置	将 `index_repository` 限制在该目录之内（解析符号链接与 `..` 之后判定）；面向不可信调用方（如多租户部署）时建议设置
`CBM_LOG_LEVEL`	按角色	最低日志级别，取值 `debug`、`info`、`warn`、`error`、`none`；轻量前端默认 `warn`，守护进程默认 `info`；标准输出始终保留给 MCP JSON-RPC
`CBM_WORKERS`	自动检测	并行索引工作线程数，范围 1 至 256；容器内检测到宿主 CPU 数而非配额时手动指定
`CBM_MEM_BUDGET_MB`	自动检测	显式的内存图预算上限（MiB），优先于推导默认值；用于将预算压到 cgroup 限额之下以留出余量
`CBM_DIAGNOSTICS`	`false`	置为 `1` 或 `true` 启用周期性资源快照与内存轨迹（见 11.6 节）
`CBM_DOWNLOAD_URL`	GitHub Releases	更新下载地址，用于测试或自托管部署
`CBM_DUMP_VERIFY_MIN_RATIO`	`0.5`	索引后比对持久化与内存节点数；低于该比例且提交数大于 50 时，`index_repository` 返回 `degraded` 而非 `indexed`；设为 `0` 禁用

注意守护进程的环境继承规则：守护进程自有组件使用的环境取自**第一个启动守护进程的会话**，后续会话加入该进程而不能替换这些值；需要修改时，应关闭全部守护进程会话、在各客户端配置中一致地更新环境变量，再重启会话。`CBM_ALLOWED_ROOT` 属于会话级设置。自定义缓存目录的用法：

```
export CBM_CACHE_DIR=~/my-projects/cbm-data
```

11.3 自定义文件扩展名

框架特有的扩展名（如 Laravel 的 `.blade.php`、ES 模块的 `.mjs`）可通过 `extra_extensions` 映射到受支持的语言。配置分两级：项目级文件为仓库根目录的 `.codebase-memory.json`，全局文件为 `~/.config/codebase-memory-mcp/config.json`（或 `$XDG_CONFIG_HOME` 下的对应路径），项目级覆盖全局中的同名冲突项。

```
// .codebase-memory.json（项目级）
{"extra_extensions": {".blade.php": "php", ".mjs": "javascript"}}
```

每个条目把一个以点号开头的扩展名映射为语言名；语言名大小写不敏感，可用的语言名清单（含别名）见仓库 README 的 Custom File Extensions 一节。未知语言名或不以点号开头的条目会被跳过并记录警告。

11.4 索引资源限制

除 11.1 节的两个上限外，索引管线具备内存感知能力：热点数据保留在内存中，必要时将提取结果溢出到磁盘，索引完成后释放内存。配合 `CBM_MEM_BUDGET_MB` 与 `CBM_WORKERS`，可以在容器配额受控的环境中稳定运行。超限行为的设计取向是保守：宁可使索引尝试失败并保留可服务的旧索引，也不发布不完整的图。

11.5 安装完整性与安全验证

每个发行版发布前经过多层验证：全部可执行候选经 VirusTotal 扫描（结论随发布件发布）；构建过程生成 SLSA 3 级来源证明（可用 GitHub CLI 核验）：

```
gh attestation verify <文件> --repo DeusData/codebase-memory-mcp --signer-workflow DeusData/codebase-memory-
mcp/.github/workflows/_build.yml
```

同时附带 Sigstore cosign 无键签名、发布件 `checksums.txt` 的 SHA-256 校验（两个安装脚本在解包前自动执行）与 CodeQL 静态安全扫描（存在未处理告警时阻断发布）。结合 2.4 节的 Defender 误报说明：遇到杀毒软件报出 `Trojan:Script/Wacatac.B!ml` 一类结论时，先比对 `checksums.txt` 中的 SHA-256 再决定是否信任。

11.6 诊断与性能问题反馈

CBM 不收集遥测，因此性能问题需要使用者自行取证。在第一个守护进程会话启动前设置 `CBM_DIAGNOSTICS=1`，复现问题后，守护进程会在系统临时目录下创建专有的随机目录，其中：

文件	内容
`trajectory.ndjson`	内存轨迹：每 5 秒一行，含 `rss`、`committed`、峰值、页错误、文件描述符与查询计数；约 8 MB 轮转；服务器退出后仍保留。内存与泄漏报告的关键证据
`snapshot.json`	仅最新一次快照，用于快速查看；正常退出时移除

确切路径由 `cbm-daemon.log` 中的 `diagnostics.start` 事件记录。轨迹只含资源计数器，不含源码与查询文本，可直接附在 Issue 中。若守护进程已在运行，需先关闭全部守护进程会话使其退出，再改变该设置。

11.7 常见故障排查

问题	处理方法
客户端的 `/mcp` 面板中看不到服务器	确认 MCP 配置中的 `command` 为绝对路径；重启智能体；用 `echo '{}'`
`index_repository` 失败	传入绝对路径
`index_repository` 在客户端超时	以 `async: true` 启动，再以 `status: true` 轮询（见 3.4 节）
`trace_path` 返回 0 条结果	先用 `search_graph(name_pattern=".*部分名称.*")` 找到精确名称
查询返回了错误项目的结果	显式添加 `project` 参数；项目名以 `list_projects` 输出为准
安装后找不到可执行文件	将安装目录加入 PATH（见 2.6 节）
界面未加载	确认以 `--ui=true` 启动；检查 `http://localhost:9749`
CBM Lens 提示 server unreachable 或 Architecture 尚不可用	确认 `?project=` 中的项目名与索引名完全一致；从项目菜单重新选择；查看 System 日志或 `${CBM_CACHE_DIR}/logs/ui.log`

11.8 重置环境

两种重置手段及其后果需要区分：删除缓存目录 `rm -rf ~/.cache/codebase-memory-mcp/` 会移除**全部项目索引与配置**，各项目需重新索引，源码不受影响；`uninstall -y --delete-indexes`（见 2.8 节）在卸载二进制与配置的同时删除全部索引。二者均不可逆，执行前确认不再需要本地图谱数据。

第 12 章 综合实战：大型陌生仓库的代码考古

前述章节分别介绍了单个工具；真实工程任务需要把它们串成一条工作流。本章以一个综合场景演练：使用者接手一个包含后端服务、部署清单与文档的多服务仓库，需要在有限时间内建立全局认识、定位一项功能的实现、评估一次待提交的改动，并把结论沉淀为团队资产。

12.1 场景与准备

假设仓库 `orders-platform` 包含若干 Go 服务、Kubernetes 部署清单、Helm 图表与 Markdown 文档。准备工作依次为：确认 CBM 已安装（第 2 章）、确认编码智能体已接入（第 4 章）、在仓库根目录执行索引。为让后续示例可复现，本节统一使用项目名 `orders-platform`（即 `list_projects` 返回的名称）。

12.2 阶段一：建立全局视图

先以 `get_architecture` 获取概览，明确语言构成、包结构、路由与热点；需要系统结构投影时单独请求 `system_structure`。若使用 CBM Lens，可直接打开 Architecture 工作区的 Overview 与 System structure 视图，用热点定位频繁被引用的区域。此阶段的目标是形成「系统由哪些部分组成、彼此如何连接」的骨架认识，而不是记住细节。

12.3 阶段二：定位功能实现

以一个具体功能为目标（例如「订单取消后退款如何触发」），先用语义搜索按含义查找：

```
search_graph(semantic_query="refund after order cancellation", semantic_limit=10)
```

再用结构化搜索交叉验证，并用 `get_file_outline` 掌握候选文件的结构、`get_code_snippet` 读取关键函数源码。文档若被索引，`Section` 节点与 `MENTIONS` 边会把相关说明与实现连在一起；同一查询中可能同时返回设计文档与代码。

12.4 阶段三：追踪调用链

对定位到的入口函数执行入站追踪，确认它被谁调用；再执行出站追踪，理清它触发的后续流程：

```
trace_path(function_name="CancelOrder", direction="inbound", depth=3)
trace_path(function_name="CancelOrder", direction="outbound", mode="data_flow")
```

若取消动作最终落到另一个服务的 HTTP 接口，`mode="cross_service"` 会沿 `HTTP_CALLS` 边跨仓库延伸；在多仓库场景中，先按 10.2 节运行跨仓库连接，链路才能到达对端项目。

12.5 阶段四：评估一次改动的影响

修改 `CancelOrder` 的签名之前，先评估影响面：

```
detect_changes()
```

该调用将工作区的未提交变更映射为受影响符号清单与风险分级；传入 `scope: "tests"` 时附带测试影响，指出哪些测试会执行到本次改动。若改动已完成提交，可对两个快照执行 `compare_graphs`，核对节点与边的增删是否符合预期。据此决定修改方案，或先补充测试。

12.6 阶段五：接入部署视角

改动完成后，回答「这次改动何时、由谁送到生产环境」：以 `run` 模式从服务出发确认容器运行的程序与镜像，以 `deploy` 模式从 CI 任务出发确认部署链条。结合 Helm 相关边（9.4 节），可以沿 HelmRelease → Chart → Deployment → Image 一路核对到源码。结论应在 CBM Lens 的 Architecture 面板或源码中复核证据（9.5 节）。

12.7 工作流复盘与最佳实践

实践	理由
查询无结果时先 `check_index_coverage` 再下结论	干净的覆盖结果只表示无记录缺口，空结果不等于不存在
追踪前先用 `search_graph` 确认符号精确名称	名称不完整是追踪空结果的最常见原因
大仓库首次索引用异步模式	避免客户端超时取消任务（3.4 节）
涉及部署结论时回到源码证据核对	基础设施追踪是静态关系，动态取值与模板可能造成缺口
团队快照按固定节奏提交	每次保存都提交会使 Git 历史膨胀（10.1 节）
删除「死代码」前人工核对动态分发与框架约定	静态图对间接调用不可见（6.4 节）
图谱结论用于辅助判断，不替代测试与审查	CBM 是结构化分析后端，不执行代码（1.5 节）

12.8 延伸资源

- 项目主页与 README：`https://github.com/DeusData/codebase-memory-mcp`
- 仓库内专题文档：`docs/CONFIGURATION.md`（配置参考）、`docs/cbmignore.md`（忽略规则）、`docs/SMART_CI.md`（Smart CI 与测试选择）、`docs/INDEX_RESOURCE_LIMITS.md`（索引资源限制）、`docs/MEASURING_SAVINGS.md`（在自己的工作负载上测量收益）
- 技术预印本：arXiv:2603.27277（令牌消耗与工具调用的测量）
- 社区：项目 Discord 服务器（链接见仓库 README）

完成本章后，读者应能独立完成从索引建立、检索定位、链路追踪、影响评估到部署核对的完整分析闭环。CBM 的工具面仍在快速演进，具体参数与行为以所用版本的官方文档为准。